

8.3.5 Implementierung der Datenkapselung

Wie Sie bereits in den Kapiteln 8.2.7 und 8.2.8 gelernt haben, sollten Sie das Prinzip der **Datenkapselung** beachten: um fehlerhaftes Manipulieren von Attributen zu vermeiden, wird der Zugriff auf diese von „außerhalb“ der Klasse gesperrt und ein kontrollierter Zugriff über setter und getter Methoden ermöglicht.



Merke:

Attribute und Methoden können als privat markiert werden. Private Attribute und Methoden können nur **innerhalb der Klassendefinition** direkt adressiert werden und können somit nicht mehr von außen manipuliert werden. Attribute und Methoden, die nicht als privat markiert sind, sind automatisch öffentlich.

Dazu werden in der Klassendefinition dem Bezeichner privater Attribute **zwei** Unterstriche **__** vorangestellt:

`self.attribut = wert` 🌐 Öffentliches Attribut
`self.__attribut = wert` 🔒 Privates Attribut

Beispiele:

`self.__x = 5` ✅ Interne Zugriffe sind weiterhin gestattet
`held.__x = 5` ❌ Zugriff von außen ist gesperrt

Im UML-Klassendiagramm werden öffentliche Elemente mit **+** und private mit **-** markiert.

Klassenbezeichner
+ attribut1 : typ
- attribut2 : typ
- attribut3 : typ
...
Klassenbezeichner(Parameterliste)
- methode1(...)
+ methode2(...)
+ methode3(...)
...

attribut1 ist öffentlich
 attribut2 und attribut3 privat

methode1 ist privat
 methode2 und methode3 öffentlich

Der Konstruktor muss nicht als öffentlich deklariert werden.

Aufgabe 1

- a) Laden Sie **Level 40**. Die Anzeige funktioniert, aber das Level kann nicht abgeschlossen werden.
- b) Betrachten Sie jetzt das folgende Klassendiagramm des Helden und setzen Sie die entsprechenden Attribute auf privat. Dies sollte Level 40 lösen, aber der Held fehlt.

Held
- x : Integer - y : Integer - richtung : String + name : String - typ : String - weiblich : Boolean
Held (xp,yp,richtung,w) + geh() + links() ...

- c) Laden Sie **Level 41**. Der Held kann nicht mehr angezeigt werden, weil kein Zugriff auf seine Position mehr möglich ist. Testen Sie dies einmal selbst, indem Sie in der `schueler.py` zuerst den Namen des Helden (öffentliches Attribut) ausgeben und dann versuchen, die x-Koordinate des Helden auszugeben. Testen Sie dabei auch, ob der Schutz durch Verwendung von Unterstrichen umgangen werden kann.
- d) Leeren Sie Ihr Programm in der `schueler.py`. Um den Zugriff auf die Attribute weiter zu ermöglichen, implementieren wir jetzt **getter** für alle privaten Attribute, da es in diesem Fall keinen Grund gibt, den lesenden Zugriff zu verhindern. Ergänzen Sie zunächst den getter für die x-Koordinate und führen Sie das Programm aus.

```
def get_x(self):  
    return self.__x
```

- e) Im Objektinspektor sollte die x-Koordinate jetzt ausgelesen werden können. Ergänzen Sie nach der gleichen Namenskonvention die getter für alle privaten Attribute. Wenn Sie alles richtig gemacht haben, wird der Held wieder angezeigt und das Level gelöst.
- f) Ergänzen Sie in der `schueler.py` ein kleines Programm, das die Position des Helden in der folgenden Form ausgibt:

„Namenloser Held steht an Position (3,2).“

- g) Leeren Sie die `schueler.py` wieder und laden Sie **Level 42**. Sie können das Level noch nicht abschließen, weil Ihre Geh- und Drehmethoden noch nicht angepasst wurden. Auch beim Zugriff auf private Attribute innerhalb der Klasse müssen die Unterstriche angebracht werden. Ergänzen Sie daher die Unterstriche bei allen Zugriffen auf Attribute innerhalb der Methoden. Wenn Sie alles richtig gemacht haben, wird die Funktionalität wiederhergestellt und der Held kann bewegt werden.
- h) Laden Sie jetzt abschließend **Level 43**. Das Framework erwartet noch einen **setter** für das Richtungsattribut. Fügen Sie zunächst einen setter für das Namensattribut hinzu:

```
def set_name(self, name):  
    if len(name)<20:  
        self.name = name  
    else:  
        print("Name zu lang!")
```

- i) Ergänzen Sie einen setter für die Richtungsangabe und stellen Sie dabei sicher, dass nur gültige Richtungsangaben übergeben werden.

Übungsaufgabe: Lassen Sie auch Richtungen der Form "S", "W", "O" und "M" zu, wobei diese natürlich zur richtigen Form korrigiert werden müssen.

Reflexionsfrage:

Aus welchem Grund muss der Konstruktor nicht als öffentliche Methode deklariert werden?

Aufgabe 2

- a) Implementieren Sie die Klasse **Hindernis** nach dem folgenden Klassendiagramm vollständig in die `hindernis.py` im Klassenordner, um **Level 44** zu lösen.
- b) Implementieren Sie die Klasse **Zettel** nach dem folgenden Klassendiagramm vollständig in der `zettel.py` im Klassenordner, um **Level 45** zu lösen.

Hindernis
- x : Integer - y : Integer - typ : String
Hindernis (x,y : int, art : str) + get_x() : int + get_y() : int + get_typ() : str + set_typ(art : str)

Zettel
- x : Integer - y : Integer - typ : String - spruch : String
Zettel (x,y : int) + get_x() : int + get_y() : int + get_typ() : int + get_spruch() : int + set_spruch(s : str)

Hinweise zu Hindernis:

Ein Hindernis kann als Typ Baum, Busch oder Berg sein. Dies wird in Form eines Strings mit dem Parameter art übergeben. Es muss natürlich entsprechend geprüft werden, dass nur gültige Arten gesetzt werden.

Bei einem Zettel gibt es keine Einschränkungen für den Zauberspruch. Dieser wird auf einen beliebigen „Standardspruch“ ihrer Wahl gesetzt.

Zusätzliche Aufgaben

- Ergänzen Sie beim Hindernis eine private Methode `randomize()`, welches die Art des Hindernis zufällig setzt. Im Konstruktor wird dann diese Methode verwendet, um eine zufällige Art zu setzen, unabhängig vom Parameter (sinnvollerweise sollte die Methode aber nur aufgerufen werden, wenn ein ungültiger Wert übergeben wurde)
- Ergänzen Sie beim Zettel eine Methode `zufalls_spruch()`, der einen zufälligen Zauberspruch erzeugt